

mdstats FINAL-GPU1 Workstation Qualification Runbook

mdstats development

2026-08-16

CURRENT RELEASE HANDOFF - FINAL-GPU1 v3

This revision-81 runbook is the current one-shot FINAL-GPU1 v3 handoff for `mdstats 0.20.214a0`. The earlier 0.20.209a0/v2 bundle is superseded and must not be used to authorize this release.

Purpose

This runbook executes the **single consolidated FINAL-GPU1 handoff** for `mdstats 0.20.214a0` / architecture revision 81. The development package does **not** contain a positive GPU result. FINAL-GPU1 becomes positive only after the supplied workstation produces release-matched CUDA/CuEquivariance evidence and the fail-closed reducer passes.

The final authority preserves the scientific baseline and phase separation established by earlier gates:

- source inference, DATA6 authority, pseudolabel authority, and source evaluation remain e3nn unless the optional CUEQ-PHASE2 path independently passes;
- pure CuEq training requires CUEQ-PHASE1;
- the historical direct six-head CuEq probes are measured but do not block the phase-separated production design when they remain negative;
- generated campaign defaults are **never** changed by FINAL-GPU1 itself.

1. Bundle contents

The workstation bundle contains:

- the exact `mdstats-0.20.214a0-complete-source-package-replay-unify1e.zip` release archive;
- locked MACE-MH-1 and MACE-MPA-0-medium foundation models;
- the LTA target-training corpus and TRUE_DFT replay corpus;
- the offline dependency/source archive supplied for the project;
- this runbook and bundle SHA-256 manifest; and
- a convenience preflight launcher.

Do not replace either foundation model. FINAL-GPU1 requires the complete SHA-256 identities encoded in `mdstats.FINAL_GPU1_LOCKED_FOUNDATION_SHA256`.

2. Runtime prerequisite

Activate the final CUDA environment intended to authorize the release. It must contain the release-matched MACE/e3nn stack plus the CuEquivariance core, Torch integration, and Torch operations distribution selected for the installed CUDA/PyTorch runtime. FINAL-GPU1 preflight records the exact installed distributions, CUDA devices, driver-visible state, deterministic settings, and module provenance.

Do not modify package source or campaign scientific settings to make preflight pass. A missing or incompatible accelerator component is a failed/deferred runtime, not permission to fall back silently.

3. Verify and unpack

From the unpacked workstation bundle root:

```
sha256sum -c VERIFY_BUNDLE_SHA256.txt
mkdir -p work/source
unzip -q mdstats-0.20.214a0-complete-source-package-replay-unify1e.zip -d work/source
cd work/source/mdstats-0.20.214a0
```

The release archive itself remains unchanged after extraction because its SHA-256 is part of every FINAL-GPU1 evidence registration.

4. Run release/model/runtime preflight

```
python tools/run_mlff_final_gpu_qualification.py preflight \
--mh1-model ../../inputs/mace-mh-1.model \
--mpa0-model ../../inputs/mace-mpa-0-medium.model \
--release-archive ../../mdstats-0.20.214a0-complete-source-package-replay-unify1e.zip \
--output ../../final-gpu1/preflight.json
```

Before expensive GPU work, inspect `final-gpu1/preflight.json`. `qualification_state=ready_for_final_gpu_execution` requires:

1. both locked foundation SHA-256 identities;
2. CUDA availability through the active PyTorch runtime;
3. a positive CUEQ-DEP1 runtime freeze; and
4. the exact release-archive SHA-256 binding.

A deferred state is not a release failure by itself; it means the positive accelerator campaign has not yet been authorized on this environment.

5. Initialize the immutable handoff root

```
python tools/run_mlff_final_gpu_qualification.py init \
--root ../../final-gpu1/run-001 \
--mh1-model ../../inputs/mace-mh-1.model \
--mpa0-model ../../inputs/mace-mpa-0-medium.model \
--release-archive ../../mdstats-0.20.214a0-complete-source-package-replay-unify1e.zip
```

This creates the 18-item matrix in `final-gpu1_handoff.json`. The run root is immutable at the registration level: do not re-run `init` on a populated root and do not replace an already registered gate result. Use the `record` command so the evidence file hash, release identity, schema, content digest, and CUEQ runtime digest (where required) are captured consistently. Replacement evidence belongs in a new run root.

6. Freeze CUEQ-DEP1 once

Capture the authorizing accelerator runtime **once** and reuse its `content_digest` across all CuEq qualification evidence:

```
mkdir -p ../../final-gpu1/run-001/raw
python tools/capture_mlff_cueq_dep1_runtime.py \
--supplied-artifact ../../inputs/mace-mh-1.model \
--supplied-artifact ../../inputs/mace-mpa-0-medium.model \
--output ../../final-gpu1/run-001/raw/cueq_dep1_runtime.json
```

Then register it:

```
python tools/run_mlff_final_gpu_qualification.py record \
--root ../../final-gpu1/run-001 \
```

```
--gate CUEQ_DEP1_RUNTIME_FREEZE \
--evidence ../../../../final-gpu1/run-001/raw/cueq_dep1_runtime.json \
--disposition auto
```

If this gate is negative, stop accelerator qualification and preserve the evidence. Do not switch to an unrecorded CuEq distribution or different CUDA environment inside the same run root.

7. Execute the qualification matrix

Use the existing campaign/benchmark authorities to produce the complete evidence matrix. The production campaign configuration remains scientifically frozen; FINAL-GPU1 is an external execution-and-reduction harness rather than a new generated campaign backend policy.

7.1 Must-pass release blockers

All nine must pass:

Gate ID	Required evidence
CUEQ_DEP1_RUNTIME_FREEZE	positive <code>CueqDep1RuntimeRecord.v1</code> from step 6
E3NN_BASELINE_COMPLETE_CAMPAIGN	complete optimized MH-1/omat_pbe/e3nn production-representative baseline with PERF-CERT1 telemetry and hard-decision fingerprints
SIZE_FIDELITY1_EXHAUSTIVE_CALIBRATION	exhaustive full-ladder/frozen-seed GPU calibration satisfying the existing SIZE-FIDELITY1 authority
PERF_P2R_WHOLE_FUNNEL_GPU_PERFORMANCE	complete whole-funnel GPU execution across the frozen SIZE-FIDELITY1 parameter grid with exact pause/resume and selection equivalence
VRAM1_PERF_P4_ACCELERATOR_MEMORY_THROUGHPUT	real accelerator VRAM/headroom/OOM/backoff and bounded-pipeline throughput evidence under the existing safety policy
CUEQ_PHASE1_TRAINING_ONLY_QUALIFICATION	short paired e3nn/CuEq trajectories plus at least one representative full pair on the same CUEQ-DEP1 runtime
SIZE_FIDELITY2_MV_SURVIVOR_REQUALIFICATION	exact q=4.8 survivor-fidelity matrix from the frozen SIZE-FIDELITY2 execution plan, with final GPU status passed
TARGET_DATA2C_MVMIGRATE1_LEARNING_CONTROLS	paired legacy-v4 versus MV learning controls at the MVQUAL1-frozen control sizes, with final GPU status passed
PERF_CERT1_END_TO_END_CERTIFICATION	authoritative baseline plus admissible accelerated profiles; exact hard decisions and a strictly positive end-to-end speedup for any recommendation

For the campaign itself, use the normal staged CLI rather than bypassing gates:

```
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> doctor
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> prepare
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> preflight
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> train
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> evaluate
python tools/mdstats-mlff-campaign.py --config <frozen-campaign.toml> verify
```

The e3nn baseline and CuEq-training realization must share the same frozen scientific inputs/protocol identities. Only the execution realization may differ where the corresponding phase authority allows it.

CUEQ-PHASE1 final assembly is performed with:

```
python tools/qualify_mlff_cueq_phase1.py qualify \
--runtime ../../../../final-gpu1/run-001/raw/cueq_dep1_runtime.json \
--short-pair <short-pair-assessment.json> \
--full-pair <full-pair-assessment.json> \
--output ../../../../final-gpu1/run-001/raw/cueq_phase1_qualification.json
```

Additional `--short-pair` and `--full-pair` arguments may be supplied for the frozen matrix. The pair-assessment files are produced by `qualify_mlff_cueq_phase1.py` pair from matched e3nn-reference and CuEq-candidate trajectory records.

7.2 Assemble SIZE-FIDELITY2 and MVMIGRATE1 typed evidence

Do not hand-edit either migration prerequisite. Assemble the exhaustive survivor-fidelity report from the campaign-frozen execution plan and its GPU checkpoints:

```
python tools/qualify_mlff_size_fidelity2.py \
--execution-plan <size_fidelity2_execution_plan.json> \
--checkpoint <checkpoint-1.json> \
--checkpoint <checkpoint-2.json> \
--output ../../../../final-gpu1/run-001/raw/size_fidelity2_qualification.json
```

Repeat `--checkpoint` for the complete frozen matrix. Then assemble the paired legacy-v4/MV learning-control rows and report:

```
python tools/qualify_mlff_target_mv_learning_control.py row <row-arguments> \
--output <mv-control-row.json>
python tools/qualify_mlff_target_mv_learning_control.py assemble \
--qualification <target_multi_view_qualification.json> \
--row <mv-control-row-1.json> \
--row <mv-control-row-2.json> \
--output ../../../../final-gpu1/run-001/raw/target_mv_learning_control.json
```

Both reports are typed release authorities. FINAL-GPU1 v3 retains the v2 typed checks and rejects generic JSON with a `passed` field, rejects a report whose content digest differs from the registered evidence, and requires both reports to name the same dataset. Register them under their exact gate IDs before reduction.

7.3 Measure-only optimization evidence

These six measurements must be present, but a negative result is admissible when the optimization remains disabled/superseded:

- PREC3_REAL_CUEQ_ACTIVATION
- MH1_ACCEL1_CUEQ_NUMERICAL_PARITY
- MH1_DATA6_1_CUEQ_DESCRIPTOR_SELECTION_PARITY
- MH1_TRAIN1_CUEQ_TRAINING_REALIZATION
- MH1_CERT1_GENERATED_DEFAULT_CUEQ_MATRIX
- PERF_P5_ACCELERATOR_PERSISTENCE_REUSE

Record the actual result as `pass` or `fail`; do not convert a negative measurement into `not_applicable`. FINAL-GPU1 requires a content-addressed evidence artifact for every measure-only item.

7.4 Optional capability evidence

These do not block the core final release:

- CUEQ_PHASE2_SELECTED_HEAD_SOURCE_EXECUTION_OPTIONAL
- MH1_DEPLOY1_MLIAP_EXPORT_AND_LAMMPS_RUNO

If PHASE2 is executed, its e3nn reference and EXTRACT1 selected-head/CuEq candidate must use the same frozen development corpus and CUEQ-DEP1 runtime. Assemble it with `tools/qualify_mlff_cueq_phase2.py` and preserve the resulting qualification JSON.

If PHASE2 is intentionally not executed, emit the explicit fail-closed optional record instead of inventing a success or leaving PERF-CERT1 without an input:

```
python tools/qualify_mlff_cueq_phase2.py deferred \  
  --runtime ../../../../final-gpu1/run-001/raw/cueq_dep1_runtime.json \  
  --output ../../../../final-gpu1/run-001/raw/cueq_phase2_deferred.json
```

Use that deferred record as `--phase2` when assembling PERF-CERT1. PHASE2 remains optional and does not become a must-pass requirement.

8. Register each result

For every completed matrix item:

```
python tools/run_mlff_final_gpu_qualification.py record \  
  --root ../../../../final-gpu1/run-001 \  
  --gate <GATE_ID> \  
  --evidence <gate-result.json> \  
  --disposition auto
```

Use `--cueq-runtime-digest <digest>` for every CuEq-dependent item whose producer record does not already expose the runtime digest in a field recognized by the registrar. The runtime freeze, PHASE1, PHASE2, and PERF-CERT1 schemas are recognized automatically; direct CuEq measurement records such as PREC3/MH1-ACCEL1/MH1-DATA6-1/MH1-TRAIN1/MH1-CERT1 normally require the explicit option. Missing runtime binding fails closed.

Check progress and byte integrity at any time:

```
python tools/run_mlff_final_gpu_qualification.py status \  
  --root ../../../../final-gpu1/run-001  
python tools/run_mlff_final_gpu_qualification.py verify \  
  --root ../../../../final-gpu1/run-001
```

`verify` re-hashes the release archive, foundation models, registration records, and copied evidence artifacts and also checks matrix/record consistency. The run root is resumable but registered gate evidence is immutable. Preserve failed evidence and logs rather than deleting them; corrected/replacement evidence belongs in a new run root.

9. Assemble PERF-CERT1

After CUEQ-PHASE1 is positive and the authoritative baseline/accelerated profile records are complete:

```
python tools/qualify_mlff_perf_cert1.py assemble \  
  --phase1 ../../../../final-gpu1/run-001/raw/cueq_phase1_qualification.json \  
  --phase2 <phase2-qualification-or-explicit-failed-record.json> \  
  --baseline <e3nn-baseline-profile.json> \  
  --candidate <phase1-accelerated-profile.json> \  
  --output ../../../../final-gpu1/run-001/raw/perf_cert1_qualification.json
```

PHASE2 is optional: a failed/non-authorizing PHASE2 record may be supplied without invalidating an admissible PHASE1 profile. PERF-CERT1 only recommends an accelerated profile if it preserves the frozen scientific authority and has total wall time strictly below the e3nn baseline.

Register the PERF-CERT1 JSON with the `PERF_CERT1_END_TO_END_CERTIFICATION` gate.

10. Final reduction

After all must-pass and measure-only evidence is registered, run an explicit integrity pass first:

```
python tools/run_mlff_final_gpu_qualification.py verify \  
  --root ../../../../final-gpu1/run-001
```

Only then reduce:

```
python tools/run_mlff_final_gpu_qualification.py reduce \  
  --root ../../../../final-gpu1/run-001 \  
  --runtime ../../../../final-gpu1/run-001/raw/cueq_dep1_runtime.json \  
  --perf-cert1 ../../../../final-gpu1/run-001/raw/perf_cert1_qualification.json \  
  --size-fidelity2 ../../../../final-gpu1/run-001/raw/size_fidelity2_qualification.json \  
  --mv-learning-control ../../../../final-gpu1/run-001/raw/target_mv_learning_control.json \  
  --output ../../../../final-gpu1/run-001/FINAL_GPU1_QUALIFICATION.json
```

A positive `FINAL_GPU1_QUALIFICATION.json` requires all of the following simultaneously:

- exact foundation-model identities;
- positive CUEQ-DEP1 runtime;
- same release archive across all evidence;
- same CUEQ runtime across all bound accelerator evidence;
- all must-pass items positive;
- all measure-only items measured and content-addressed;
- positive PERF-CERT1 with matching PHASE1/PHASE2/runtime content digests;
- typed, passing SIZE-FIDELITY2 and MVMIGRATE1 learning-control records whose registered content digests match exactly;
- no cross-release or cross-runtime provenance contamination; and
- a clean pre-reduction handoff-integrity re-hash with no post-registration byte changes.

11. Interpretation

If FINAL-GPU1 passes, `authorization.recommended_profile_id` reports the PERF-CERT1 recommendation. `authorization.generated_default_change_authorized` remains `false`. If PERF-CERT1 recommends an accelerated profile, `generated_default_policy_revision_required=true` indicates that a **separate versioned policy/default-migration gate** is required before generated campaign defaults may change.

If FINAL-GPU1 fails, retain `FINAL_GPU1_QUALIFICATION.json`, the handoff manifest, records, evidence, and logs. The authoritative e3nn scientific path remains the fallback; a failed optional/measure-only optimization does not acquire scientific authority by omission.

12. Atomic TARGET-DATA2C v5 activation after a pass

A passing current-release FINAL-GPU1 v3 record authorizes the MVMIGRATE1 transaction but does not mutate campaign state by itself. First reconstruct the exact replacement generation without writes:

```
python tools/activate_mlff_target_mv_migration.py \  
  --config <frozen-campaign.toml> \  
  --final-gpu1 ../../../../final-gpu1/run-001/FINAL_GPU1_QUALIFICATION.json
```

The dry-run must report `dry_run_passed`. Then publish the generation explicitly:

```
python tools/activate_mlff_target_mv_migration.py \  
  --config <frozen-campaign.toml> \  
  --final-gpu1 ../../../../final-gpu1/run-001/FINAL_GPU1_QUALIFICATION.json \  
  --apply
```

The apply step is one SQLite transaction. It preserves the historical TARGET-DATA2C v4 ladder, publishes the authorized fixed-eight v5 ladder and fresh TARGET-DATA2D v3 convergence plan, stores the two typed final-GPU records plus FINAL-GPU1 qualification, invalidates stale TARGET-DATA2E/prepare aliases, and writes a content-addressed activation receipt. It refuses to replace an existing activation with a different receipt.

REPLAY-UNIFY1 GPU pseudo-label execution

FINAL-GPU1 v3 adds `REPLAY_UNIFY1_GPU_PSEUDOLABEL_EXECUTION` as a must-pass runtime-bound gate. Run a release-matched `foundation_pseudolabel` campaign `prepare` against the supplied 12,000-frame replay source with the locked foundation model/head and frozen CuEq runtime. Evidence must show exactly 10,000 train and 2,000 monitor members, identical pseudo/true monitor geometry membership, finite energy/force/stress predictions, successful authenticated cache restart with zero reinference, and recorded batch throughput/peak VRAM. Register the resulting content-addressed report under this exact gate ID before reduction.